

Task 4: AI Hand Gesture Computer Control System

Submitted By:	G. Pavana Deepika
Task ID:	Task 4 - Computer Vision & OS Automation
Tech Stack:	Python, OpenCV, MediaPipe, PyAutoGUI
Target Domain:	Real-Time Touchless Operating System Control

1. Introduction & Project Overview

Building upon standard static computer vision classification, this project implements an active **AI Hand Gesture Computer Control System**. By utilizing a standard webcam input, the system continuously tracks human hand landmarks and translates precise spatial gestures into operating system-level commands, allowing users to control cursor movement, clicking, and scrolling without physical contact with peripherals.

2. Project Objectives

The primary objective of this task is to construct a robust, real-time automation pipeline that bridges computer vision hand tracking with OS-level event execution:

- **Real-Time Cursor Navigation:** Map index finger landmark coordinates directly to monitor screen resolution for smooth pointer movement.
- **Touchless Clicking Mechanism:** Detect finger distance thresholds (e.g., thumb-to-index pinch) to trigger mouse click operations programmatically.
- **System Control Integration:** Maintain low latency and high accuracy across frame updates to ensure practical usability.

3. Technology Stack & Libraries

- **Python:** Core programming language utilized for application orchestration and script execution.
- **OpenCV (cv2):** Handles live webcam feed capture, image processing transformations, and real-time GUI frame rendering.
- **MediaPipe Hands:** Google's cross-platform machine learning framework optimized for high-fidelity 3D hand landmark coordinate estimation.

- **PyAutoGUI:** Cross-platform GUI automation library used to programmatically control mouse cursor positioning, clicking, and system actions.

4. Pipeline & Implementation Methodology

The execution workflow is structured into continuous real-time iterative blocks:

- **Frame Capture & Mirroring:** Reads sequential frames from `cv2.VideoCapture(0)` and flips them horizontally for intuitive mirror-image visualization.
- **Color Space Conversion:** Transforms input BGR video frames into RGB color format as required by the MediaPipe Hands processing pipeline.
- **Landmark Extraction:** Detects 21 3D hand joint coordinates on processed frames to identify key tracking points (such as Index Finger Tip and Thumb Tip).

5. Feature Extraction & Control Logic

The system evaluates coordinate thresholds and geometric distance formulas across landmark joints to execute system controls:

Gesture / Action	Target Landmarks	Heuristic Rule / Condition	Behavioral Description
Cursor Movement	Index Finger Tip (ID 8)	Normalized X & Y coordinates scaled to screen width and height.	Moves the mouse pointer smoothly across the monitor following finger motion.
Click Trigger	Index Tip & Thumb Tip (IDs 8 & 4)	Euclidean distance $d < 35$ pixels between thumb and index tips.	Executes a programmatic mouse click event with built-in debounce delay.

6. Step-by-Step Execution Guide

To run the application locally in your environment:

```
# 1. Install required dependencies
pip install opencv-python mediapipe pyautogui

# 2. Place script file (controller.py) inside your task folder
# 3. Execute application via terminal
python controller.py
```

7. Results & Visual Verification

The application successfully renders real-time skeleton annotations on the live video feed while simultaneously controlling OS cursor position and click events based on physical hand motions.

Status: Completed & Verified